

Project 1 NetLogo

Part 1 — Study of the Software

What are NetLogo's elements?

NetLogo is built around a very small number of core concepts, and everything else in the language is built on top of them. The central idea is that a model is made of **agents**: independent entities that each hold their own state and follow their own instructions. There are four kinds of agents in NetLogo.

- **Turtles**: mobile agents that move around the world, have a position (xcor, ycor), a heading, a color and a shape. They represent whatever the model needs to be an active, moving entity — a person, an animal, a termite, a car...
- **Patches**: the fixed cells that make up the grid of the world. Every patch has its own coordinates (pxcor, pycor) and its own color (pcolor); patches never move, but they can hold information (food, pheromone, terrain type, etc.).
- **Links**: connections between two turtles, used to represent relationships or networks (not used directly in this project, but part of the core agent model).
- **The Observer**: not really an "agent" living in the world, but the entity that oversees the whole simulation. The Observer cannot move or have a color; it can only give orders to the other agents using ask, or perform world-level actions such as clearing everything.

On top of these four agent types, a NetLogo model is organized around a few other essential building blocks: **procedures** (blocks of instructions written with to ... end, either Observer procedures or agent/turtle procedures), **global and agent-owned variables** (globals for values shared by the whole model, turtles-own for values that belong to each turtle individually), and **interface widgets** (buttons, sliders, switches, monitors and plots) that let a person interact with the running model without touching the code.

What can you do with NetLogo?

NetLogo is a language and environment designed specifically for **agent-based modeling**, meaning it lets you build a simulation from the bottom up: instead of writing one big formula that describes the whole system, you describe the simple behavior of a single agent, and then let many copies of that agent interact. The overall, often complex, behavior of the system emerges naturally from these many simple individual interactions, rather than being explicitly programmed.

- Simulate natural phenomena: flocking birds, ant colonies, termite nest-building, predator-prey dynamics, diffusion of a substance, spread of a disease or a fire.
- Simulate social and economic phenomena: traffic jams, segregation patterns in a city, the spread of information or opinions, market dynamics.

- Teach and learn core ideas in computer science and complex systems: loops, conditionals, randomness, recursion, and above all the idea of emergence — how organized, seemingly "intelligent" behavior can appear from agents that individually follow very simple, local rules.
- Prototype and test hypotheses quickly, thanks to sliders and buttons that let a user change parameters live and immediately see the effect on the simulation, without touching the code.

In short, NetLogo turns the question "what happens if many simple things interact?" into something a student can actually build, run, watch, and modify within a single afternoon.

What are NetLogo's windows (tabs)?

A NetLogo project is organized into three main tabs, each with a very distinct purpose:

- **Interface tab:** the visual tab, where the world (the black grid of patches and turtles) is displayed, and where all interactive widgets live — buttons (to run procedures), sliders (to adjust numeric variables live), switches, choosers, monitors (to display a value) and plots (to chart values over time). This is the tab a user interacts with while a simulation is running.
- **Info tab:** a documentation tab, meant to describe the model in plain language, structured into standard sections such as WHAT IS IT?, HOW IT WORKS, HOW TO USE IT, THINGS TO NOTICE, THINGS TO TRY, and so on. It is meant to let someone who has never seen the code understand what the model does and how to use it.
- **Code tab:** where the actual NetLogo program is written, as a series of procedures (to setup ... end, to go ... end, etc.). This is the only tab where the underlying logic of the model is actually defined.

In addition to these three tabs, the **Command Center**, always visible at the bottom of the Interface tab, lets the user type single NetLogo commands directly and execute them immediately, without writing a full procedure — either from the *observer* context, or from a *turtle* / *patch* context, which changes who is being addressed by the command being typed.

List and role of NetLogo commands that I used :

NetLogo commands can be grouped by who is allowed to use them: the Observer, a turtle, or a patch. Below is a summary of the commands actually used throughout this project, and the role each one plays.

Observer / world commands

Command	Role
clear-all (ca)	Erases everything: all turtles, all patches (reset to black), all plots and drawings. Used to fully reset a model at the start of setup.
clear-turtles (ct)	Erases only the turtles; patches and their colors are left untouched.
create-turtles n (crt n)	Creates n new turtles, placed by default at random headings at the center of the world.
create-ordered-turtles n (cro n)	Same as crt, but the turtles are numbered in order and evenly spread in heading.
ask <agentset> [...]	The central command of NetLogo: tells every agent in the given set (turtles, patches, or a single agent) to execute the instructions inside the brackets, each one individually.
reset-ticks / tick	reset-ticks initializes the tick counter to 0 at the end of setup; tick advances it by one at the end of each cycle of go, driving plots and the ticks display.
Command	Role

random n	Returns a random whole number between 0 and n-1; used everywhere to introduce chance into a model's behavior.
one-of <agentset>	Randomly picks a single agent from a set (for example, one random patch).
n-of k <agentset>	Randomly picks k agents from a set (used to scatter a fixed number of colored patches).

Turtle commands

Command	Role
forward n (fd n)	Moves the turtle forward by n steps in the direction it is currently facing.
right n / left n (rt / lt)	Turns the turtle's heading by n degrees, clockwise or counter-clockwise.
setxy x y	Instantly places the turtle at exact coordinates (x, y), without animating the move.
move-to <patch or turtle>	Instantly moves the turtle to the center of the target patch or turtle.
set color <color>	Changes the turtle's own display color.
pen-down / pen-up	Lowest or raises the turtle's pen; while down, any movement leaves a trail on the patches behind it.
patch-here / patch-ahead n	Refers to the patch the turtle is currently standing on, or the patch n steps ahead in its current heading.

Patch commands

Command	Role
set pcolor <color>	Changes the color of a patch; used to represent state (empty, food, a pile, etc.).
ask patches with [...] [...]	Filters the set of patches by a condition, then applies an instruction to only the matching ones.
patches	Refers to the full set of every patch in the world, most often combined with with, n-of or one-of.

Plot commands

Command	Role
plot <value>	Adds the given numeric value as the next point on the currently active plot pen; without this keyword, a value is computed but never drawn.
plot-pen-down / plot-pen-up	Starts or stops a given pen from drawing on the plot.

Part 2 — What I Did and What I Understood

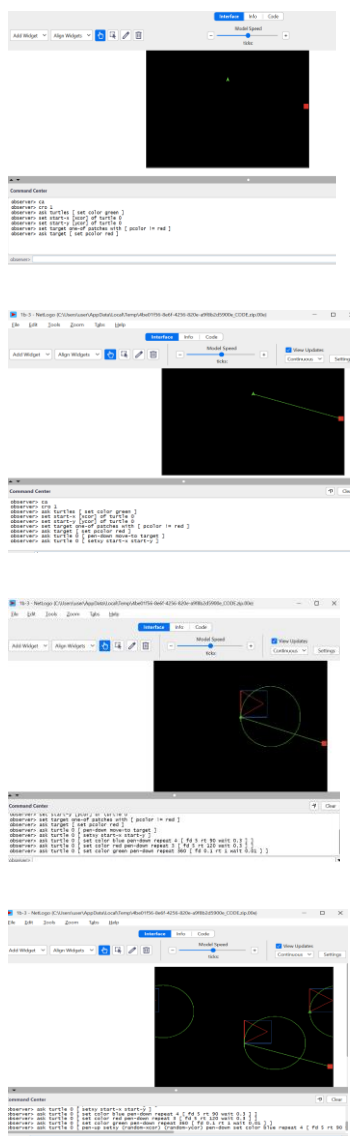
Exercises 1 to 3 — Agents, patches, and the Observer *(Please check the “Info” tab for the documentation and explanations of the NetLogo model.)*

The first exercises were about getting comfortable with the most basic vocabulary of NetLogo: creating a turtle, giving it a color, coloring a patch, and moving the turtle onto that patch. The key idea I understood here is that the **Observer never acts directly on the world** — it can only give orders to agents through ask. Every time I wanted a turtle to move or change color, I had to write ask turtle 0 [...], never a bare command like fd 5 on its own, because commands like fd or set color only make sense from the point of view of a turtle, not of the Observer, which has no position, heading or color of its own.

Exercise 3 made this distinction explicit by repeating the exact same actions as Exercise 2, but typing them directly into the Command Center in the Observer context, always wrapped in `ask turtle 0 [ ... ]`. This showed me that the visual result is strictly identical whether the logic lives inside stored procedures (Exercise 2) or is typed live, one command at a time (Exercise 3) the difference is purely about who is giving the order and how the instructions are entered, not about what actually happens in the simulation.

I also learned the difference between **clear-turtles (ct)**, which only removes turtles, and **clear-all (ca)**, which resets the whole world including patch colors – an important distinction as soon as a model actually uses colored patches. Similarly, I understood the purpose of the pen: **pen-down** makes any subsequent turtle movement leave a visible trail, which is what lets a path be drawn on screen, and this trail is always the same color as the turtle itself, since NetLogo has no separate "pen color" variable. Drawing geometric shapes (a square, a triangle, a circle) with repeat also taught me a small but important piece of geometry: turning by exactly 360 divided by the number of sides, repeated that many times, always brings the turtle back to its original heading, which is why the shape closes perfectly.

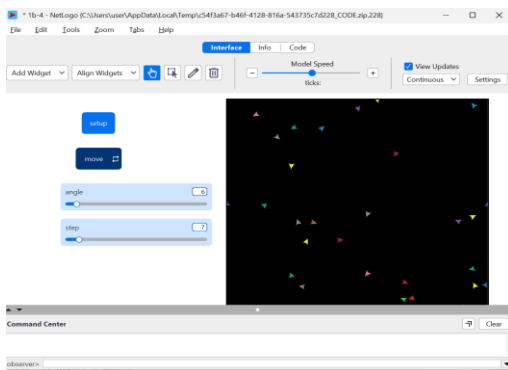
EX03 With Command Center



Exercise 4 — Interactive control with sliders

This exercise introduced the idea that a model does not have to have its parameters hard-coded in the Code tab. By creating sliders named `angle` and `step` in the Interface, and simply referring to `angle` and `step` by name inside the code, NetLogo automatically treats the sliders as global variables that can be read directly from any procedure. I understood

that this is what makes a model genuinely interactive: while the forever button move keeps re-running the movement rule tick after tick, I can drag the sliders live and immediately see the effect — a small angle keeps the motion almost straight, while a large angle makes it chaotic and jittery; a small step keeps the group clustered, while a large step spreads it out quickly. I also understood that the built-in Model Speed control is completely different in nature: it only changes how fast NetLogo redraws the screen, never the actual rules governing the agents' behavior.



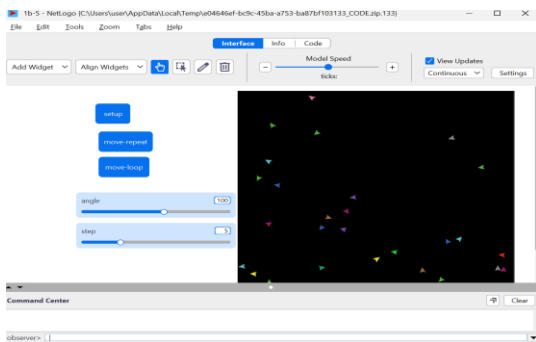
- **30 tortues** colorées, dispersées sur l'écran après setup.
- **angle = 6°**, **step = 7** : à chaque tick, chaque tortue tourne d'un angle **fixe** et faible (6°), puis avance d'une distance **fixe** de 7 pas — seule la direction (gauche ou droite) est tirée au hasard.

Exercise 5 — True random walk, and repeat versus loop

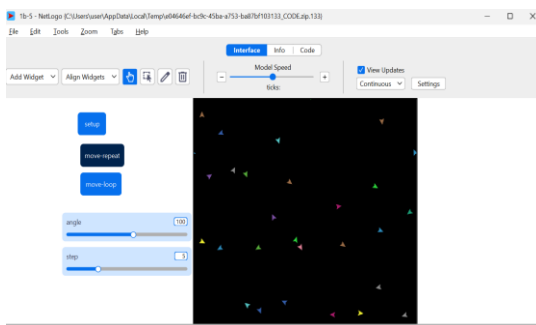
Exercise 5 pushed the idea of randomness further: instead of a fixed direction and a fixed distance, both the turning angle and the distance moved became random on every single step, capped by the angle and step sliders. This is the classic definition of a **random walk**, a model used far beyond NetLogo to describe things like Brownian motion or an animal's search for food.

The most valuable understanding from this exercise came from comparing **repeat** and **loop**. `repeat n [ ... ]` always executes its block exactly *n* times and then stops on its own — it is the right tool when the number of repetitions is known in advance. `loop [ ... ]`, on the other hand, has no built-in limit at all: it will run forever unless the code inside it explicitly calls stop when some condition becomes true. I used this to make agents perform a random walk until they physically reached the edge of the world, which repeat alone could not express, since the number of steps needed to reach the edge is not known in advance and differs for every single agent.

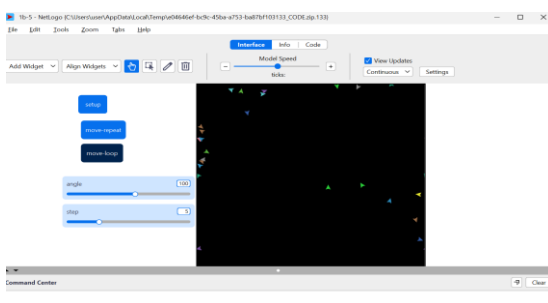
I also ran into, and resolved, a subtle but important pitfall: placing `wait` inside a loop that is itself inside an `ask turtles [ ... ]` block makes NetLogo run through each turtle's entire loop one after another, completely sequentially, instead of advancing every turtle in parallel — so a single turtle finishes its whole journey to the edge before the next one even starts moving. Adding a `display` command right after each step, inside the loop, forces NetLogo to redraw the view at that exact moment, which restores the visible, gradual animation I expected. This taught me that in NetLogo, correctness of the final result and correctness of the visual animation while it happens are two different things, and a loop that is logically fine can still look wrong on screen without an explicit refresh.



1/SETUP : 30 tortues de couleurs différentes apparaissent, dispersées à des positions aléatoires sur l'écran. Les sliders sont réglés sur angle = 100 et step = 5. Le compteur ticks est encore vide.



move-repeat



move-loop

	<i>move-repeat</i>	<i>move-loop</i>
Every moment	all the turtles at once	one turtle at a time
Pace	synchronized, step-by-step, for the entire group	sequential: one turtle completes its entire journey before the next one begins.
End of the movement	after exactly 50 steps, for everyone at the same time	when this turtle reaches the edge — then we move on to the next one
Final position	scattered, anywhere (no connection to the edges)	always close to an edge (due to the stop condition)

move-repeat

	Low value	High value
low angle (e.g., 10)	The turtles move forward in an almost straight line → after 50 steps, they will have moved a considerable distance from their starting point, in a fairly consistent direction.	////
high angle (e.g., 100, as in your screenshots)	/////	Each turtle changes direction radically at every step → the movements partially cancel each other out → after 50 steps, it remains close to its starting area, as observed in your images.
low step (e.g., 1–2)	Each step is tiny → even after 50 repetitions, there is little net displacement.	////////
high step (e.g., 15–20)	////////	Each step is large → even with a steep angle that creates a zigzag pattern, the overall spread will be wider, and the turtles will cover more ground.

move-loop

Setting	Effect
low angle	More direct path to an edge → the turtle reaches it faster (fewer steps required).
high angle	Zigzag path → the turtle takes much longer to reach an edge because it "goes around in circles" locally instead of moving forward efficiently.
low step	Since each step is small, it takes more steps (and therefore more time) to reach the edge.
high step count	Since each step is large, the edge is reached more quickly, in fewer steps.

Exercise 6 — Termites: a real case of emergence

This final exercise brought together everything from the previous ones into a genuine multi-agent system, and it is the one that taught me the most about what NetLogo, and agent-based modeling in general, are really for. The scenario: ten termites, each following an extremely simple cycle — wander randomly until finding a wood chip, pick it up, wander randomly again until finding an existing pile (or another isolated chip), drop the chip next to it, then move slightly away before starting the cycle again. No termite ever knows how many piles exist, where the biggest one is, or what any other termite is doing.

What struck me while testing this model is that, despite this total absence of any global plan or communication between agents, the piles do not stay scattered: over time, they merge into fewer and fewer, larger and larger piles. I understood the mechanism behind this: a larger pile simply occupies more physical space, which mechanically increases the chance that a wandering termite bumps into it rather than into a smaller, isolated chip — so bigger piles attract more deposits, which makes them grow even faster, in a kind of snowball effect that eventually concentrates almost everything into very few piles. This is precisely what **emergence** means: a coherent, organized, almost "designed-looking" outcome, produced entirely by agents that individually know nothing beyond their immediate surroundings.

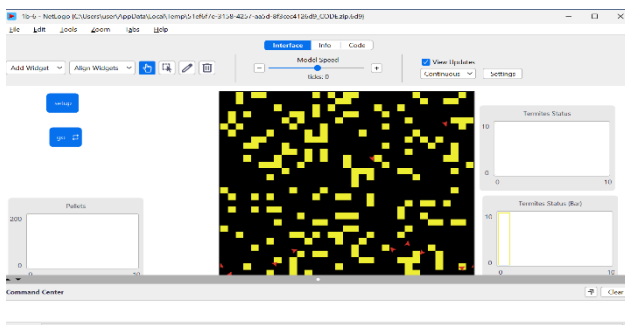
Building this model also forced me to debug two very instructive mistakes. First, an early version only let a termite recognize an existing pile by the color green, but since no patch is ever green at the very start of the simulation, the very first termite carrying a chip could never find anywhere to deposit it, and got stuck in an infinite recursive search — freezing the whole model, with the tick counter stuck at zero, without producing any visible error. The fix was to let a lone yellow wood chip count as a valid "pile" of size one, exactly as the exercise statement said ("piles are only a single object at the beginning"), which allows the very first deposit to happen and the whole process to bootstrap itself. Second, the plots stayed empty even though the tick counter was correctly advancing, because the pen commands computed a value (for example count patches with [pcolor = yellow]) without ever actually calling plot on it — a value that is calculated but never explicitly plotted is simply discarded, silently, with no error message.

Both bugs were a good lesson in how NetLogo can fail completely silently, and how important it is to check the tick counter and the state of the world directly, through the Command Center, rather than assuming a lack of visible error means everything is working.

Interpretation

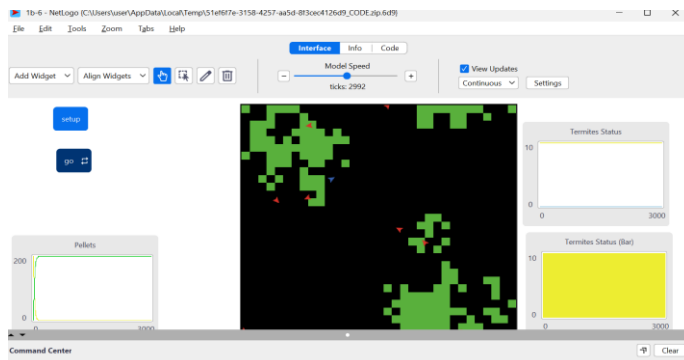
After setup (ticks: 0)

- 10 red termites appear, scattered randomly, none of them carrying a wood chip yet.
- 200 yellow patches (wood chips) are scattered individually across the whole grid — no pile exists yet.
- The plots (Pellets, Termites Status) are empty, since no tick has occurred yet.



After go (ticks: 2992)

- The yellow wood chips have almost entirely disappeared, replaced by several green piles clustered at different locations on the screen.
- Only a few termites are still visible outside the piles (most of them are busy inside the piles).
- Pellets plot: the green curve ("organized") has risen to about 200, confirming that nearly all isolated wood chips have been absorbed into piles.
- Termites Status (Bar) plot: a large yellow bar → at this moment, most termites are searching for a wood chip, very few are carrying one.



General conclusion

Across these six exercises, I moved from simply telling one turtle where to go, to building a population of independent agents whose collective behavior I could no longer predict just by reading the rule of a single one of them. What I take away most from this project is the central idea of agent-based modeling itself: complex, organized, sometimes surprising global behavior does not require a global plan — it can emerge purely from many agents each following simple, local, and often randomized rules. I also came away with a much more concrete understanding of NetLogo's own architecture: the strict separation between the Observer and the agents, the role of ask as the only bridge between them, the difference between commands that only make sense at the world level (clear-all, tick) and those that only make sense from an agent's point of view (forward, set color, pen-down), and the practical difference between a fixed number of repetitions (repeat) and an open-ended one governed by a condition (loop). Finally, debugging the termite model taught me that in this kind of environment, silent failures are the norm rather than the exception, and that verifying assumptions directly in the Command Center is often faster and more reliable than reasoning about the code alone.